



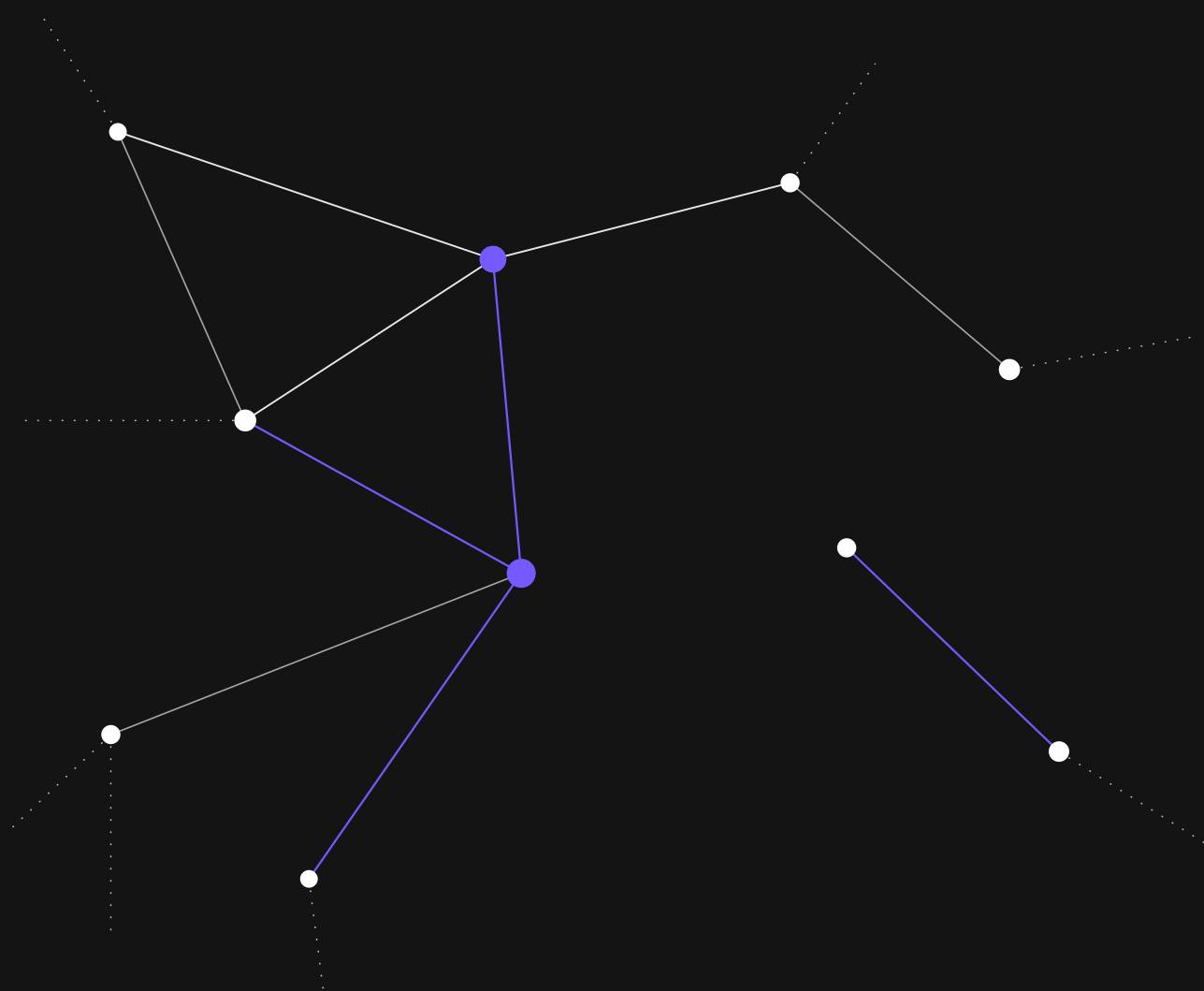
Машинное обучение

Лонгрид

Глубокое обучение

05

Графовые нейронные сети



Содержание

1. Постановка задачи	2
1.1. Графовое представление данных	2
1.2. Типы задач на графах	2
1.3. Трансдуктивная и индуктивная постановки	3
1.4. Инвариантность и эквивариантность к перестановкам	4
2. Бейзлайны без обучаемой передачи сообщений	5
2.1. Модель без учета структуры графа	5
2.2. DeepWalk	5
2.3. Агрегация признаков соседей	6
2.4. Общая схема графовой модели	6
3. Передача сообщений в графовых нейронных сетях	7
3.1. Общая схема передачи сообщений	7
3.2. Пример вычисления одного слоя	8
3.3. Глубина модели и рецептивное поле	8
4. Базовые архитектуры графовых нейронных сетей	9
4.1. Graph Convolutional Network	9
4.2. GraphSAGE	10
4.3. Graph Attention Network	10
4.4. Сравнение GCN, GraphSAGE и GAT	11
4.5. Вычислительная сложность	11
5. Предсказание и функции потерь	12
6. Агрегация представлений для задач на уровне графа	12
7. Гомофилия и гетерофилия в графах	13
8. Самосупервизорное обучение графовых нейронных сетей	15
9. Современные направления	15
9.1. Фундаментальные модели для графов	15
10. Основные выводы	16
A. Pointer Network и REINFORCE	17
A.1. Постановка задачи	17
A.2. Архитектура Pointer Network	17
A.3. Обучение с учителем	18
A.4. Минимизация ожидаемой стоимости	19
A.5. Контрольная переменная и оценка градиента	19
A.6. Вычислительные свойства и ограничения	20
Дополнительная литература	21

1. Постановка задачи

1.1. ГРАФОВОЕ ПРЕДСТАВЛЕНИЕ ДАННЫХ

Во многих задачах важны не только признаки отдельных объектов, но и связи между ними. В молекуле объектами служат атомы, а связями - химические связи между ними. В графе цитирования объектами являются статьи, а ребро показывает, что одна статья ссылается на другую. В социальной сети вершины соответствуют пользователям, а ребра - их взаимодействиям. Граф позволяет описать объекты и структуру связей между ними в одной модели.

Формально граф задается парой

$$G = (V, \mathcal{E}),$$

где $V = \{1, \dots, n\}$ - множество вершин, а $\mathcal{E}$ - множество ребер. Для ориентированного графа матрица смежности $A \in \{0, 1\}^{n \times n}$ определяется равенством

$$A_{vu} = 1 \iff (u, v) \in \mathcal{E}.$$

Здесь первый индекс соответствует вершине-получателю, а второй - вершине-отправителю. В неориентированном графе ребро между u и v задает сразу два элемента: $A_{vu} = A_{uv} = 1$, поэтому $A = A^T$. В списке ребер такого графа каждую пару далее записываем один раз.

Признаки вершин собраны в матрицу

$$X = [x_1^T; \dots; x_n^T] \in \mathbb{R}^{n \times d_0}.$$

Если ребра имеют признаки, каждому ребру (u, v) соответствует вектор $e_{uv} \in \mathbb{R}^{d_e}$. Целевая переменная Y может быть задана для вершин, ребер или графа целиком. Требуется построить параметрическую модель

$$f_{\theta}(A, X, \{e_{uv}\}),$$

выход которой зависит от признаков объектов и структуры связей. Графовая модель нужна тогда, когда ответ определяется не только отдельными объектами, но и тем, как они связаны. Например, две статьи с похожим текстом могут получить разные метки из-за того, на какие работы они ссылаются и какие статьи ссылаются на них.

1.2. ТИПЫ ЗАДАЧ НА ГРАФАХ

Задачи на графах различаются прежде всего тем, для какого объекта нужно построить предсказание: для каждой вершины, для пары вершин или для всего графа.

Если ответ требуется для каждой вершины, модель строит n выходов:

$$f_{\theta}^{\text{node}}(A, X) \in \mathbb{R}^{n \times c}.$$

Например, в графе цитирования можно классифицировать статьи по темам, а в сети взаимодействий - определять тип каждого пользователя.

Для задачи на ребрах дополнительно задается список пар вершин, которые нужно оценить. В него

могут входить наблюдаемые ребра и пары-кандидаты. Зафиксируем порядок этих пар:

$$Q = ((u_1, v_1), \dots, (u_{m'}, v_{m'})).$$

Модель строит по одному выходу для каждой пары из этого списка:

$$f_{\theta}^{\text{edge}}(A, X, Q) \in \mathbb{R}^{m' \times c},$$

где m' - число оцениваемых пар. К этому типу задач относятся предсказание наличия связи между двумя вершинами и определение типа уже существующей связи. Например, можно предсказывать, появится ли связь между двумя пользователями, или классифицировать отношение между двумя объектами сцены.

Если требуется предсказать свойство всего графа, модель строит один выход:

$$f_{\theta}^{\text{graph}}(A, X) \in \mathbb{R}^c.$$

Примеры - предсказание свойства молекулы и классификация графа взаимодействий.

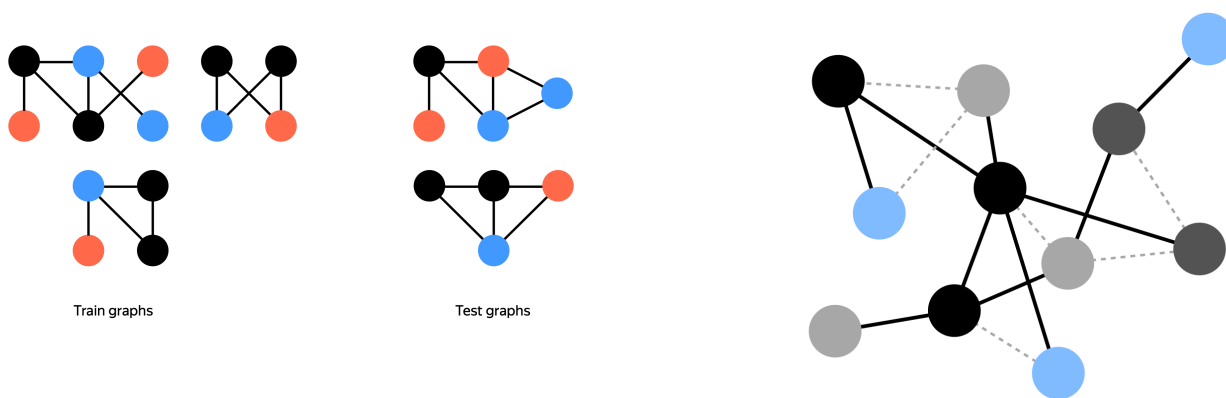


Рисунок 1. Слева - задача на графе целиком. Справа - предсказание недостающих ребер. В задаче на вершинах модель строит отдельный выход для каждой вершины.

1.3. ТРАНСДУКТИВНАЯ И ИНДУКТИВНАЯ ПОСТАНОВКИ

Здесь важно, на каких объектах модель будет применяться после обучения. В трансдуктивной постановке модель обучается и делает предсказания для объектов одного фиксированного графа. Множество вершин задано заранее, но часть меток или ребер, которые требуется предсказать, может быть скрыта. В индуктивной постановке после обучения появляются новые вершины или новые графы. Поэтому модель должна задавать общую функцию от признаков и окрестности, а не хранить отдельный обучаемый вектор для каждой вершины.

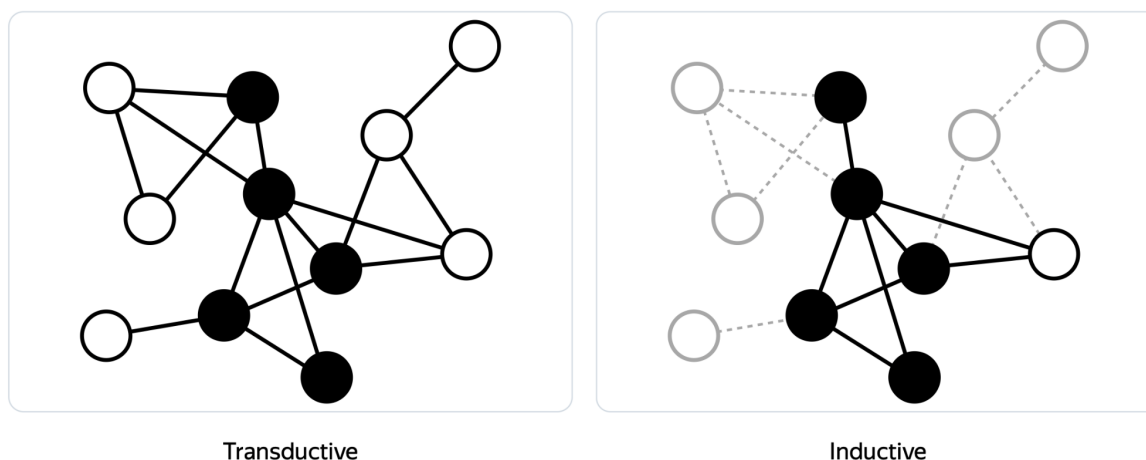


Рисунок 2. Трансдуктивная и индуктивная постановки. Серым отмечена информация, которая не использовалась при обучении.

1.4. ИНВАРИАНТНОСТЬ И ЭКВИВАРИАНТНОСТЬ К ПЕРЕСТАНОВКАМ

Номера вершин условны. Если перенумеровать пользователей в социальной сети, структура связей останется прежней. Содержательные предсказания модели не должны зависеть от того, какие номера присвоены пользователям. Пусть $P \in \{0, 1\}^{n \times n}$ - матрица перестановки. После перенумерации

$$A' = PAP^T, \quad X' = PX.$$

Для задачи на вершинах требуется эквивариантность:

$$f_{\theta}^{\text{node}}(PAP^T, PX) = P f_{\theta}^{\text{node}}(A, X).$$

Выходы меняют порядок вместе с вершинами. Для задачи на графе целиком требуется инвариантность:

$$f_{\theta}^{\text{graph}}(PAP^T, PX) = f_{\theta}^{\text{graph}}(A, X).$$

Для задачи на ребрах вместе с нумерацией вершин преобразуются концы каждой пары из Q и признаки соответствующих ребер. Значения выходов при этом остаются сопоставлены тем же парам.

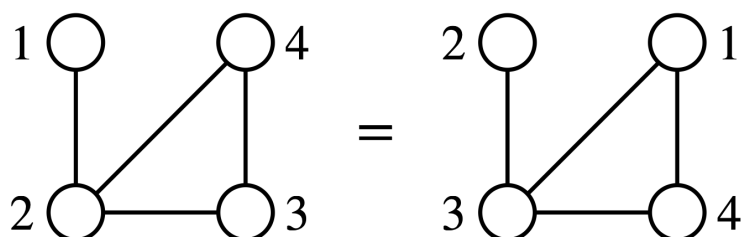


Рисунок 3. Один граф при двух нумерациях вершин. Перенумерация не изменяет структуру графа.

2. Бейзлайны без обучаемой передачи сообщений

2.1. МОДЕЛЬ БЕЗ УЧЕТА СТРУКТУРЫ ГРАФА

Начнем с модели, которая обрабатывает каждую вершину независимо от остальных. Одна и та же функция применяется ко всем строкам матрицы признаков:

$$h_v^{(0)} = x_v, \quad h_v^{(1)} = \text{MLP}_{\theta}(h_v^{(0)}).$$

Параметры общие, поэтому перенумерация вершин только переставит строки выхода. Но связей такая модель не видит: ее выход не зависит от A . Две вершины с одинаковыми признаками получают одинаковые представления, даже если их окрестности устроены по-разному.

2.2. DEEPWALK

DeepWalk сопоставляет каждой вершине обучаемый вектор z_v . Обучающие пары строятся из случайных блужданий. Из разных вершин запускаются последовательности

$$v_1, v_2, v_3, \dots, v_T,$$

а вершины, попавшие в одно контекстное окно, образуют положительные пары для skip-gram.

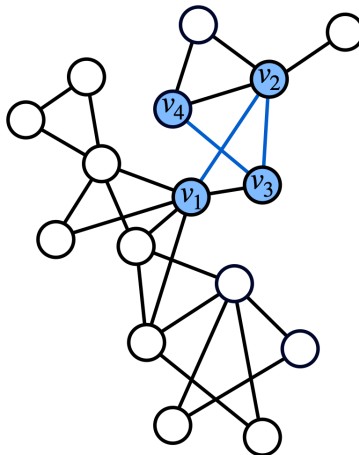


Рисунок 4. Контекстные пары вершин, полученные из случайного блуждания по графу.

В варианте с negative sampling для положительной пары (u, v) максимизируется выражение

$$\log \sigma(z_u^T c_v) + \sum_{i=1}^{K_{\text{neg}}} \log \sigma(-z_u^T c_{w_i}),$$

где c_v - вектор контекстной вершины, а $w_1, \dots, w_{K_{\text{neg}}}$ - отрицательные примеры. DeepWalk использует структуру графа, но не вычисляет представление из признаков вершины. Вектор z_v является отдельным параметром для каждой вершины, поэтому метод трансдуктивный и не переносится на новую вершину без дополнительного обучения.¹

¹ Perozzi, Al-Rfou, Skiena. DeepWalk

2.3. АГРЕГАЦИЯ ПРИЗНАКОВ СОСЕДЕЙ

Другой бейзлайн строит признаки вершины явно. Для числового признака по окрестности можно вычислить среднее, минимум и максимум, а для категориального признака - долю каждой категории:

$$z_v = [x_v \parallel \text{mean}_{u \in N(v)} x_u \parallel \min_{u \in N(v)} x_u \parallel \max_{u \in N(v)} x_u \parallel \text{freq}_{u \in N(v)} x_u].$$

После этого к z_v применяется обычный классификатор или регрессор. Параметры агрегации не обучаются. Такой вариант называют Neighborhood Feature Aggregation, или NFA.

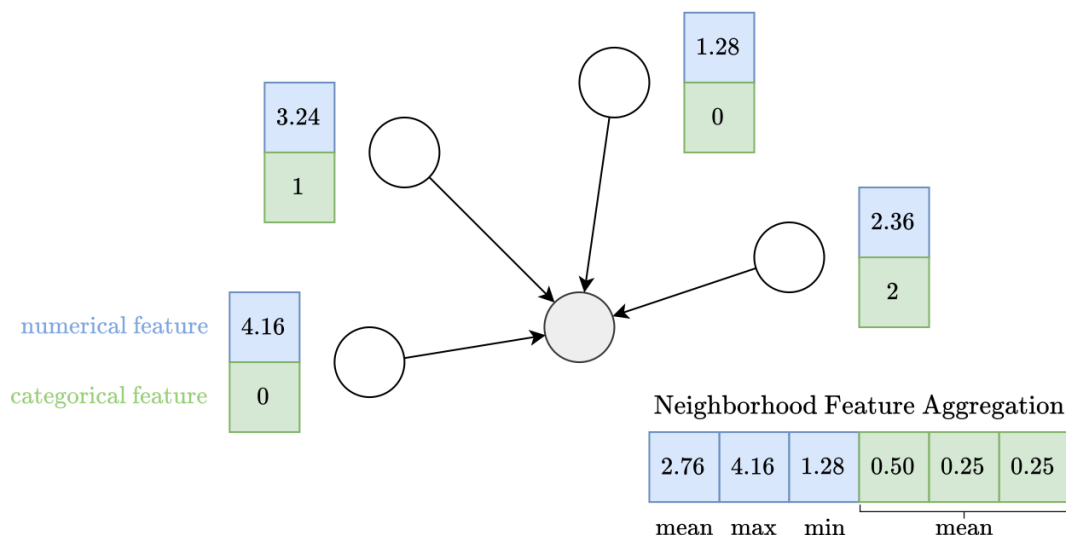


Рисунок 5. NFA для числового и категориального признаков соседних вершин.

MLP на исходных признаках, NFA и DeepWalk отвечают на разные вопросы. Первый вариант измеряет качество без структуры графа, второй проверяет пользу локальной структуры при фиксированной агрегации, третий использует структуру, но хранит отдельные параметры вершин. Их полезно сравнивать с GNN как самостоятельные бейзлайны.

2.4. ОБЩАЯ СХЕМА ГРАФОВОЙ МОДЕЛИ

Графовый кодировщик строит представления вершин. Затем вид выходного блока определяется уровнем целевой переменной: для вершины используется одно представление, для ребра - представления двух концов, для графа - инвариантная агрегация всех представлений.

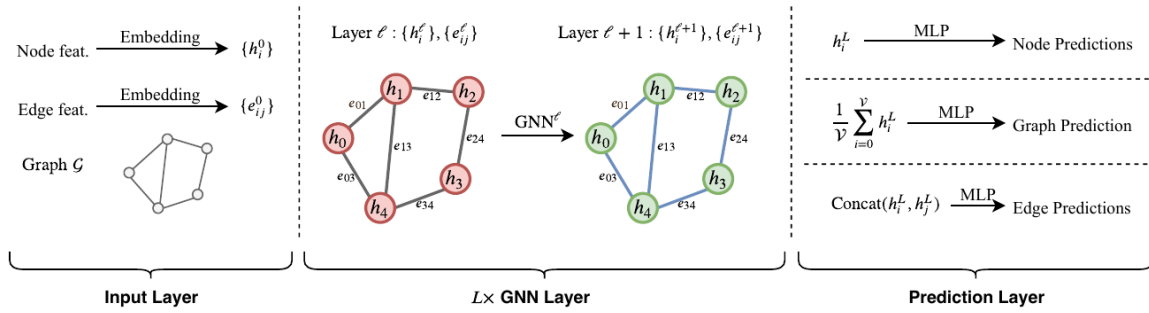


Рисунок 6. Графовый кодировщик и выходные блоки для задач на вершинах, ребрах и графах. Источник: Dwivedi et al., рис. 3, CC BY 4.0.

3. Передача сообщений в графовых нейронных сетях

3.1. ОБЩАЯ СХЕМА ПЕРЕДАЧИ СООБЩЕНИЙ

Чтобы учесть структуру графа, на каждом слое вершина получает сообщения от соседей, объединяет их в один вектор и обновляет свое представление. Число соседей у разных вершин различается, поэтому агрегация должна работать с любым числом входов и не зависеть от их порядка.

Определим входящую окрестность вершины

$$N(v) = \{u \in V : A_{vu} = 1\}.$$

Сначала для каждого ребра вычисляется сообщение от вершины u к вершине v :

$$m_{u \rightarrow v}^{(k)} = \phi_k(h_v^{(k-1)}, h_u^{(k-1)}, e_{uv}),$$

затем все входящие сообщения объединяются:

$$m_v^{(k)} = \text{AGG}_{u \in N(v)} \{m_{u \rightarrow v}^{(k)}\},$$

и полученный агрегат используется для обновления представления вершины:

$$h_v^{(k)} = \psi_k(h_v^{(k-1)}, m_v^{(k)}).$$

$m_{u \rightarrow v}^{(k)}$ обозначает сообщение по ребру, а $m_v^{(k)}$ - агрегированное сообщение для вершины v . Функции ϕ_k и ψ_k имеют общие параметры для всех вершин и ребер. Оператор AGG должен быть инвариантен к порядку аргументов. Этому условию удовлетворяют сумма, среднее и покомпонентный максимум. При этих условиях слой эквивалентен к перестановке вершин.²

² Gilmer et al. Neural Message Passing for Quantum Chemistry

3.2. ПРИМЕР ВЫЧИСЛЕНИЯ ОДНОГО СЛОЯ

Рассмотрим неориентированный граф из четырех вершин с ребрами

$$\mathcal{E} = \{(1, 2), (1, 3), (2, 4)\}$$

и одномерными признаками

$$h_1^{(0)} = 1, \quad h_2^{(0)} = 2, \quad h_3^{(0)} = 4, \quad h_4^{(0)} = 8.$$

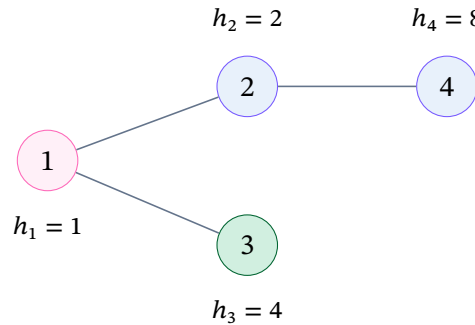


Рисунок 7. Граф для вычисления одного слоя передачи сообщений.

Пусть сообщение равно представлению отправителя, агрегация задается средним, а обновление имеет вид

$$m_v^{(1)} = \frac{1}{|N(v)|} \sum_{u \in N(v)} h_u^{(0)}, \quad h_v^{(1)} = \text{ReLU}(0.5h_v^{(0)} + m_v^{(1)}).$$

Для вершины 1

$$m_1^{(1)} = \frac{h_2^{(0)} + h_3^{(0)}}{2} = 3, \quad h_1^{(1)} = \text{ReLU}(0.5 \cdot 1 + 3) = 3.5.$$

Для остальных вершин

$$m_2^{(1)} = \frac{1 + 8}{2} = 4.5, \quad m_3^{(1)} = 1, \quad m_4^{(1)} = 2,$$

поэтому

$$H^{(1)} = (3.5, 5.5, 3, 6)^\top.$$

Уже после одного слоя представление каждой вершины зависит не только от ее собственного признака, но и от признаков соседей. Одна и та же функция обновления работает для вершин с разным числом соседей. Нормировка при этом важна: при суммировании без нормировки масштаб агрегата обычно растет вместе со степенью вершины.

3.3. ГЛУБИНА МОДЕЛИ И РЕЦЕПТИВНОЕ ПОЛЕ

Один слой передает информацию между соседними вершинами. После двух слоев вершина получает информацию от соседей своих соседей, а после K слоев - от всех вершин на расстоянии не более K ребер. Эта K -шаговая окрестность образует рецептивное поле вершины.

$$h_v^{(K)} = F_{\theta}(\{x_u : \text{dist}(u, v) \leq K\}, A).$$

Рост глубины увеличивает рецептивное поле и вычислительную стоимость. При фиксированном K модель не использует признаки вершин за пределами K -шаговой окрестности.

4. Базовые архитектуры графовых нейронных сетей

4.1. GRAPH CONVOLUTIONAL NETWORK

В этом подразделе рассматривается неориентированный граф.

Самый простой способ учесть окрестность - сложить собственное представление вершины и представления ее соседей:

$$s_v^{(k)} = h_v^{(k)} + \sum_{u \in N(v)} h_u^{(k)}.$$

Масштаб $s_v^{(k)}$ зависит от степени вершины. При одинаковом распределении признаков вершина с большим числом соседей получает агрегат большей нормы.

Для учета собственной вершины добавим петли

$$\tilde{A} = A + I$$

и определим диагональную матрицу степеней

$$\tilde{D}_{vv} = \tilde{d}_v = \sum_{u=1}^n \tilde{A}_{vu}.$$

В GCN вклад вершины u в обновление вершины v нормируется степенями обеих вершин:

$$h_v^{(k+1)} = \sigma \left(\sum_{u \in N(v) \cup \{v\}} \frac{1}{\sqrt{\tilde{d}_v \tilde{d}_u}} (W^{(k)})^T h_u^{(k)} \right).$$

В матричной форме тот же слой записывается как

$$H^{(k+1)} = \sigma \left(\tilde{D}^{-1/2} \tilde{A} \tilde{D}^{-1/2} H^{(k)} W^{(k)} \right).$$

В покомпонентной записи $h_v^{(k)}$ считается столбцом, а в матричной записи представления являются строками $H^{(k)} \in \mathbb{R}^{n \times d_k}$. Матрица $W^{(k)} \in \mathbb{R}^{d_k \times d_{k+1}}$ обучается, а σ применяется построчно.

Для графа из предыдущего примера после добавления петель $\tilde{d} = (3, 3, 2, 2)$. Коэффициент для сообщения по ребру $(1, 2)$ равен $1/3$, а для сообщения по ребру $(1, 3)$ - $1/\sqrt{6}$. Нормировка зависит от степеней отправителя и получателя.³

³ Kipf, Welling. Semi-Supervised Classification with Graph Convolutional Networks

4.2. GRAPH SAGE

GraphSAGE отдельно агрегирует окрестность и затем объединяет агрегат с собственным представлением вершины. Для варианта с усреднением

$$a_v^{(k)} = \frac{1}{|N(v)|} \sum_{u \in N(v)} h_u^{(k-1)},$$

а для изолированной вершины полагают $a_v^{(k)} = 0$. Собственное представление вершины объединяется с агрегатом:

$$h_v^{(k)} = \sigma \left(W^{(k)} [h_v^{(k-1)} \| a_v^{(k)}] \right).$$

Параметры не зависят от номера вершины. Поэтому GraphSAGE можно применять к новым вершинам и новым графам, если для них известны признаки и окрестности. В исходной работе для масштабирования использовалась выборка соседей. Это вычислительное приближение полной агрегации: оно уменьшает стоимость, но может удалить полезную часть окрестности.⁴

4.3. GRAPH ATTENTION NETWORK

В GCN веса соседей определяются степенями графа и не зависят от их признаков. Но для конкретной вершины одни соседи могут быть полезнее других. В GAT вес каждого соседа вычисляется по текущим представлениям. Сначала выполняется линейное преобразование

$$z_v = Wh_v.$$

Для вершины v и кандидата $u \in N(v) \cup \{v\}$ определяется ненормированный коэффициент

$$s_{vu} = \text{LeakyReLU} (a^T [z_v \| z_u]).$$

Локальная нормировка по окрестности вершины v имеет вид

$$\alpha_{vu} = \frac{\exp(s_{vu})}{\sum_{j \in N(v) \cup \{v\}} \exp(s_{vj})}.$$

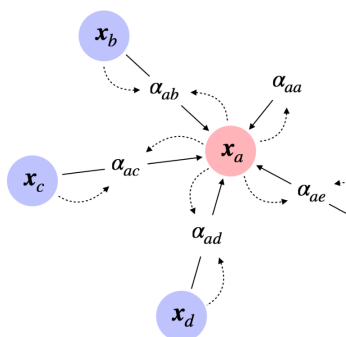
Обновленное представление равно

$$h'_v = \sigma \left(\sum_{u \in N(v) \cup \{v\}} \alpha_{vu} z_u \right).$$

При использовании нескольких головок каждая головка имеет собственные параметры и вычисляет свои коэффициенты α_{vu} . Результаты головок конкатенируются или усредняются. В исходном GAT коэффициент зависит от признаков двух вершин, но не использует e_{uv} ; включение признаков ребра является расширением модели.⁵

⁴ Hamilton, Ying, Leskovec. Inductive Representation Learning on Large Graphs

⁵ Velickovic et al. Graph Attention Networks

Рисунок 8. Коэффициенты α_{vu} вычисляются для ребер в окрестности вершины v и нормируются локальным softmax.

4.4. СРАВНЕНИЕ GCN, GRAPHSAGE И GAT

Модель	Вес соседа	Собственная вершина	Новые вершины	Ограничения
GCN	$1/\sqrt{\tilde{d}_v \tilde{d}_u}$	петля	допустимы	требуется нормировка по структуре графа
GraphSAGE	среднее по окрестности	отдельный член в конкатенации	допустимы	агрегатор задается отдельно от собственного представления
GAT	обучаемый α_{vu}	включается в локальный softmax	допустимы	коэффициент вычисляется для каждого ребра

Все три слоя используют общие параметры и поэтому не привязаны к фиксированной нумерации вершин. Они различаются правилом взвешивания соседей, способом учета собственной вершины и стоимостью обработки окрестности. Корректное экспериментальное сравнение требует одинаковых признаков, разбиения данных, числа параметров и протокола настройки.

4.5. ВЫЧИСЛИТЕЛЬНАЯ СЛОЖНОСТЬ

Разреженный граф хранится списком из $m = |\mathcal{E}|$ ребер или разреженной матрицей смежности. Для GCN линейное преобразование всех вершин и последующая агрегация имеют суммарную сложность

$$O(nd_k d_{k+1} + md_{k+1}).$$

В GraphSAGE с усреднением сначала агрегируются входные представления размерности d_k , а затем к объединению собственного представления и агрегата применяется линейный слой. С точностью до постоянного множителя сложность равна

$$O(md_k + nd_k d_{k+1}).$$

Для GAT обозначим размерность одной головки через d_h , число головок через q , а их суммарную размерность через $D = qd_h$. Линейные проекции вершин требуют $O(nd_{in}D)$ операций, вычисление коэффициентов внимания и агрегация по ребрам - $O(mD)$. Поэтому

$$O(nd_{in}D + mD)$$

операций требуется на один слой GAT.

Если весь граф не помещается в память, обучение проводят на подграфах. Способ построения мини-батча определяет, какая часть окрестности доступна модели, поэтому его нужно фиксировать при сравнении архитектур.

5. Предсказание и функции потерь

После K слоев у нас есть представления вершин, но еще нет ответа для конкретной задачи. Способ получить этот ответ зависит от того, что требуется предсказать: метку вершины, наличие ребра или свойство всего графа. Кодировщик строит матрицу

$$H^{(K)} = \text{GNN}_{\theta}(A, X, \{e_{uv}\}).$$

Для классификации вершин общий линейный слой преобразует каждую строку в логиты:

$$o_v = W_{\text{out}} h_v^{(K)} + b_{\text{out}}.$$

Пусть $V_L \subseteq V$ - множество вершин с известными метками $y_v \in \{1, \dots, c\}$. Функция потерь равна

$$\mathcal{L}_{\text{node}} = -\frac{1}{|V_L|} \sum_{v \in V_L} \log \text{softmax}(o_v)_{y_v}.$$

Параметры кодировщика и выходного слоя обучаются совместно. Построчное применение одного выходного слоя сохраняет эквивариантность к перестановкам.

Для предсказания наличия ребра задается функция пары представлений:

$$s_{uv} = g_{\omega}(h_u^{(K)}, h_v^{(K)}), \quad p_{uv} = \sigma(s_{uv}).$$

Пусть Q - список положительных и отрицательных пар, а $y_{uv} \in \{0, 1\}$ показывает, существует ли соответствующее ребро. Бинарная кросс-энтропия имеет вид

$$\mathcal{L}_{\text{edge}} = - \sum_{(u,v) \in Q} [y_{uv} \log p_{uv} + (1 - y_{uv}) \log(1 - p_{uv})].$$

Если положительная пара уже присутствует в исходном графе, соответствующее ребро удаляют из входа кодировщика на время обучения или оценки. Иначе модель непосредственно получает информацию о правильном ответе. Если для кандидатной пары доступны отдельные признаки, их можно передать в g_{ω} дополнительным аргументом. Для задачи на графе представления вершин сначала объединяются инвариантным оператором READOUT, а затем передаются в общий выходной слой.

6. Агрегация представлений для задач на уровне графа

Для предсказания свойства всего графа нужно объединить переменное число представлений вершин в один вектор фиксированной размерности. Простая конкатенация не подходит: ее размер зависит от числа вершин, а результат - от их порядка. Поэтому после K слоев передачи сообщений применяется

оператор READOUT:

$$h_G = \text{READOUT}(h_1^{(K)}, \dots, h_n^{(K)}).$$

На вход оператору поступает мультимножество: одинаковые представления разных вершин учитываются как отдельные элементы. Сохранится ли информация об их кратности в результате, зависит от выбранного READOUT. Оператор READOUT должен быть инвариантен к перестановке вершин:

$$\text{READOUT}(h_{\pi(1)}^{(K)}, \dots, h_{\pi(n)}^{(K)}) = \text{READOUT}(h_1^{(K)}, \dots, h_n^{(K)})$$

для любой перестановки π .

Для представлений размерности d базовые операторы задаются покомпонентно:

$$h_G^{\text{sum}} = \sum_{v \in V} h_v^{(K)}, \quad h_G^{\text{mean}} = \frac{1}{|V|} \sum_{v \in V} h_v^{(K)},$$

$$(h_G^{\text{max}})_j = \max_{v \in V} (h_v^{(K)})_j, \quad j = 1, \dots, d.$$

Среднее не меняется при равномерном повторении всех элементов мультимножества. Поэтому наборы $\{1, 1\}$ и $\{1, 1, 1, 1\}$ имеют одинаковое среднее. Сумма различает эти наборы, но ее масштаб зависит от числа вершин. Покомпонентный максимум сохраняет экстремальное значение каждой координаты, но не сохраняет число его повторений и значения остальных элементов.

Выбор оператора определяется свойствами целевой величины. Для аддитивной характеристики, например полной массы молекулы, используется сумма представлений атомов. Для характеристики, нормированной по числу вершин, используется среднее. После агрегации предсказание вычисляется моделью $\hat{y}_G = g_\omega(h_G)$.

7. Гомофилия и гетерофилия в графах

Передача сообщений смешивает признаки соседних вершин. В графе цитирования связанные статьи часто относятся к близким темам. В графе взаимодействий ребро, наоборот, может регулярно соединять объекты разных типов. Это различие описывают понятиями гомофилии и гетерофилии. Пусть каждой вершине v сопоставлена метка y_v . Граф называют гомофильным, если ребра преимущественно соединяют вершины с одинаковыми метками, и гетерофильным, если значительная доля ребер соединяет вершины с различными метками.

Доля ребер между вершинами одного класса равна

$$h_{\text{edge}} = \frac{|\{(u, v) \in \mathcal{E} : y_u = y_v\}|}{|\mathcal{E}|}.$$

Значение $h_{\text{edge}} = 1$ означает, что каждое ребро соединяет вершины одного класса. При $h_{\text{edge}} = 0$ таких ребер нет. Эта статистика зависит от соотношения классов: при сильном дисбалансе высокая доля ребер внутри преобладающего класса возможна без выраженной локальной гомофилии.

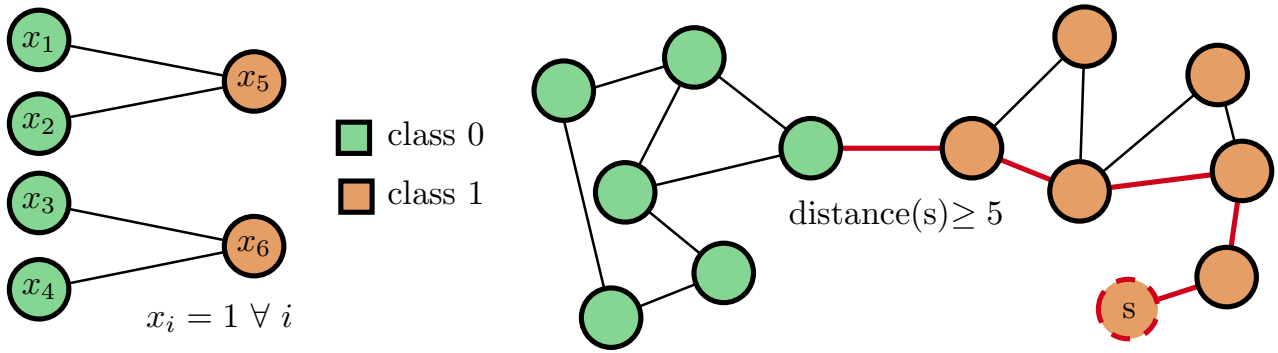


Рисунок 9. Слева показана гетерофильная задача, в которой классы различаются по степени вершины. Справа показан гомофильный граф, где ответ зависит от расстояния до выделенной вершины.

На левом рисунке признаки всех вершин одинаковы, но один слой с суммированием различает классы по числу соседей. На правом рисунке большинство ребер соединяет вершины одного класса, однако для ответа требуется информация из удаленной части графа. После K слоев представление вершины зависит только от ее K -шаговой окрестности. Если выделенная вершина s лежит за ее пределами, локальная модель не получает нужную информацию. Эти примеры показывают, что значение h_{edge} не определяет сложность задачи и качество модели однозначно. Результат также зависит от признаков вершин, целевой величины, правила обновления и радиуса передачи сообщений.⁶

В исследовании Platonov et al. были обнаружены дубликаты в наборах Squirrel и Chameleon: 2978 из 5201 вершины и 1387 из 2277 вершин соответственно. Одинаковые объекты могли попадать в обучающую и тестовую части. Удаление дубликатов и единая процедура настройки изменили сравнительные результаты моделей. Стандартные GCN, GraphSAGE и GAT оказались конкурентоспособными на исправленных версиях наборов данных. Поэтому сравнение методов для гетерофильных графов должно контролировать дубликаты, разбиение данных, признаки и процедуру настройки.⁷

Сравнение в этой работе относится не к изолированным формулам GCN, GraphSAGE и GAT, а к полным моделям. Графовый слой дополняется нормировкой, остаточной связью и полносвязным блоком:

$$Z^{(k)} = H^{(k-1)} + \text{GNN}_k(\text{LN}(H^{(k-1)})),$$

$$H^{(k)} = Z^{(k)} + \text{FFN}_k(\text{LN}(Z^{(k)})).$$

Кроме того, полезно не смешивать собственное представление вершины с соседями до преобразования. В варианте с SEP сначала вычисляется агрегат

$$a_v^{(k)} = \text{AGG}_{u \in N(v)} h_u^{(k-1)},$$

а затем собственное представление и агрегат передаются в функцию обновления отдельно:

$$\tilde{h}_v^{(k)} = \psi_k([h_v^{(k-1)} \| a_v^{(k)}]).$$

Поэтому вывод статьи состоит не в том, что гетерофилия не влияет на задачу. Он состоит в том, что специализированную архитектуру нужно сравнивать с корректно собранными стандартными слоями, где сохранен сигнал собственной вершины и используется полный блок обучения.

⁶ Arnaiz-Rodriguez, Errica. Demystifying Common Beliefs in Graph Machine Learning, Figure 2

⁷ Platonov et al. A Critical Look at the Evaluation of GNNs under Heterophily

8. Самосупервизорное обучение графовых нейронных сетей

Если внешней разметки нет, обучающую задачу можно построить из самого графа. У графа есть два разных источника сигнала: атрибуты вершин X и структура A . Восстановление только одного из них не требует от представления сохранять второй.

В GrASP маскируются и признаки части вершин, и часть ребер:

$$\tilde{X} = \text{Mask}_X(X), \quad \tilde{A} = \text{Mask}_E(A), \quad H = \text{Encoder}(\tilde{X}, \tilde{A}).$$

По общему представлению H решаются три задачи:

1. восстановить скрытые признаки вершин;
2. предсказать скрытые ребра;
3. предсказать степени вершин как дополнительную характеристику структуры.

Суммарная функция потерь имеет вид

$$\mathcal{L}_{\text{pretrain}} = \lambda_X \mathcal{L}_{\text{features}} + \lambda_E \mathcal{L}_{\text{edges}} + \lambda_D \mathcal{L}_{\text{degree}}.$$

Первое слагаемое требует сохранять атрибутивный сигнал, второе и третье - структурный. В экспериментах авторов совместная постановка дает более высокий средний ранг среди рассмотренных методов самосупервизорного обучения, чем задачи, ориентированные только на признаки или только на структуру. После предобучения кодировщик используется в целевой задаче.⁸

9. Современные направления

9.1. ФУНДАМЕНТАЛЬНЫЕ МОДЕЛИ ДЛЯ ГРАФОВ

Перенос модели между графовыми наборами данных осложнен тем, что в них различаются смысл и размерность признаков, типы ребер, целевые переменные и масштаб. Фундаментальные модели для графов пытаются построить общую схему обучения или вывода для нескольких наборов данных либо распределения задач. Единого определения и единой архитектуры для этого класса пока нет.

G2T-FM преобразует задачи классификации и регрессии на уровне вершин в представление, пригодное для табличной фундаментальной модели.⁹ Поддержка конкретного типа целевой переменной определяется используемой табличной моделью. GraphPFN проходит метаобучение на распределении синтетических графов и решает новые задачи классификации и регрессии вершин по контекстным примерам.¹⁰ Оба метода работают с задачами на уровне вершин. Предсказание ребер и задачи на уровне графа не входят в их текущую постановку.

⁸ Ereemeev et al. Capturing Structure and Feature Signals in Graph Self-Supervised Learning

⁹ G2T-FM

¹⁰ GraphPFN

10. Основные выводы

1. Графовая задача задается множеством вершин, множеством ребер, признаками и уровнем целевой переменной. Ответ может относиться к вершине, ребру или графу целиком.
2. Слой передачи сообщений состоит из функции сообщения, симметричной агрегации по соседям и функции обновления представления вершины. После K слоев представление зависит от K -шаговой окрестности.
3. GCN использует симметрично нормированную матрицу смежности, GraphSAGE отдельно агрегирует соседей, GAT вычисляет обучаемые коэффициенты для ребер в окрестности вершины.
4. Для задач на уровне графа применяется инвариантный к перестановкам оператор READOUT. Сумма, среднее и максимум сохраняют разные свойства мультимножества представлений.
5. Доля гомофильных ребер зависит от распределения классов и не определяет качество архитектуры сама по себе. Сравнение моделей требует фиксированного разбиения данных и одинакового протокола настройки.
6. MLP, NFA и DeepWalk задают разные бейзлайны без обучаемой передачи сообщений.
7. Совместное восстановление признаков и ребер обучает кодировщик сохранять атрибутивный и структурный сигналы. Фундаментальные модели для графов исследуют перенос между наборами данных и распределениями задач.

A. Pointer Network и REINFORCE

Этот раздел не относится к графовым нейронным сетям. Pointer Network рассматривается как отдельный пример модели с переменным числом допустимых действий, а REINFORCE - как способ оптимизации ожидаемой стоимости дискретного решения.

A.1. ПОСТАНОВКА ЗАДАЧИ

Пусть вход состоит из n элементов

$$x = (x_1, \dots, x_n).$$

Требуется построить последовательность индексов

$$\pi = (\pi_1, \dots, \pi_T),$$

где на шаге t индекс π_t выбирается из множества допустимых действий

$$\mathcal{A}_t(x, \pi_{<t}) \subseteq \{1, \dots, n\}.$$

Размер пространства действий зависит от длины входа, поэтому классификатор с фиксированным числом выходов для такой постановки не подходит.

В задаче коммивояжера входными элементами служат координаты городов $x_1, \dots, x_n$, а ответом является перестановка всех индексов. Для замкнутого маршрута минимизируется длина

$$L(\pi | x) = \sum_{t=1}^{n-1} \|x_{\pi_{t+1}} - x_{\pi_t}\|_2 + \|x_{\pi_1} - x_{\pi_n}\|_2.$$

Перед выбором на шаге t из множества допустимых действий исключаются уже выбранные города:

$$\mathcal{A}_t = \{1, \dots, n\} \setminus \{\pi_1, \dots, \pi_{t-1}\}.$$

Для четырех точек один из допустимых маршрутов имеет вид

$$\pi = (2, 4, 1, 3).$$

До первого выбора $\mathcal{A}_1 = \{1, 2, 3, 4\}$. После выбора $\pi_1 = 2$ множество допустимых действий равно $\mathcal{A}_2 = \{1, 3, 4\}$. На каждом следующем шаге softmax вычисляется только по еще не выбранным точкам.

A.2. АРХИТЕКТУРА POINTER NETWORK

В исходной Pointer Network последовательность входных элементов обрабатывается LSTM-кодировщиком. Для каждого элемента строится представление

$$e_i = \text{Encoder}(x_1, \dots, x_n)_i.$$

LSTM-декодировщик на шаге t формирует состояние d_t . Оценка выбора элемента i задается аддитивным вниманием:

$$u_i^{(t)} = v^T \tanh(W_e e_i + W_d d_t).$$

Для учета допустимых действий вводится маска

$$m_i^{(t)} = \begin{cases} 0, & i \in \mathcal{A}_t, \\ -\infty, & i \notin \mathcal{A}_t. \end{cases}$$

Распределение следующего индекса имеет вид

$$p_{\theta}(\pi_t = i \mid \pi_{<t}, x) = \frac{\exp(u_i^{(t)} + m_i^{(t)})}{\sum_{j=1}^n \exp(u_j^{(t)} + m_j^{(t)})}.$$

Здесь e_i - представление элемента i , d_t - состояние декодировщика, W_e , W_d и v - обучаемые параметры функции оценки, $u_i^{(t)}$ - логит выбора, а $m_i^{(t)}$ - маска допустимых действий $\mathcal{A}_t$. Число логитов равно числу элементов конкретного входа. Выходной слой выбирает индекс входного элемента, а не класс из заранее заданного словаря.¹¹

Совместное распределение над последовательностями раскладывается авторегрессионно:

$$p_{\theta}(\pi \mid x) = \prod_{t=1}^T p_{\theta}(\pi_t \mid \pi_{<t}, x).$$

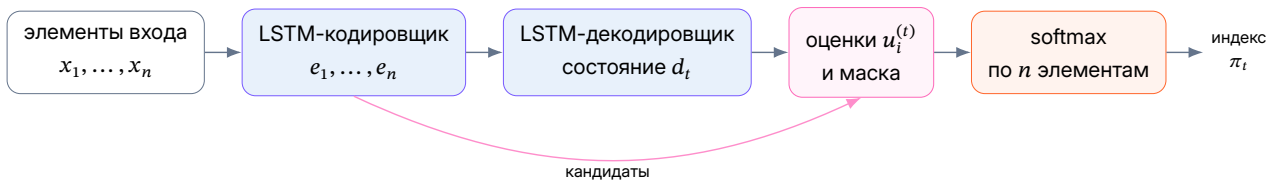


Рисунок 10. Распределение следующего индекса вычисляется по элементам входа. Маска исключает недопустимые действия.

В исходной архитектуре кодировщик является LSTM, поэтому представления зависят от порядка элементов входной последовательности. Инвариантность и эквивариантность к перестановкам из этой архитектуры не следуют. Для задач на множествах порядок входа рандомизируют либо используют кодировщик с требуемой симметрией.

А.3. ОБУЧЕНИЕ С УЧИТЕЛЕМ

Если для входа x известна целевая последовательность π^* , параметры оцениваются методом максимального правдоподобия. Функция потерь равна

$$\mathcal{L}_{\text{sup}}(\theta) = - \sum_{t=1}^T \log p_{\theta}(\pi_t^* \mid \pi_{<t}^*, x).$$

Во время teacher forcing декодировщик получает правильные предыдущие действия $\pi_{<t}^*$. Для такого обучения требуется набор целевых решений, качество которых ограничивает качество обучаемой политики.

¹¹ Vinyals, Fortunato, Jaitly. Pointer Networks

А.4. МИНИМИЗАЦИЯ ОЖИДАЕМОЙ СТОИМОСТИ

Если целевые последовательности отсутствуют, но для любого построенного решения вычисляется стоимость $L(\pi | x)$, оптимизируется математическое ожидание

$$J(\theta | x) = \mathbb{E}_{\pi \sim p_\theta(\cdot | x)} [L(\pi | x)].$$

Для дискретного пространства решений

$$J(\theta | x) = \sum_{\pi} p_\theta(\pi | x) L(\pi | x).$$

Предположим, что стоимость $L(\pi | x)$ не имеет явной зависимости от θ . Тогда

$$\begin{aligned} \nabla_\theta J(\theta | x) &= \sum_{\pi} L(\pi | x) \nabla_\theta p_\theta(\pi | x) \\ &= \sum_{\pi} L(\pi | x) p_\theta(\pi | x) \nabla_\theta \log p_\theta(\pi | x) \\ &= \mathbb{E}_{\pi \sim p_\theta(\cdot | x)} [L(\pi | x) \nabla_\theta \log p_\theta(\pi | x)]. \end{aligned}$$

Во втором равенстве использовано тождество

$$\nabla_\theta p_\theta(\pi | x) = p_\theta(\pi | x) \nabla_\theta \log p_\theta(\pi | x).$$

Полученная оценка называется оценкой на основе логарифмической производной (score-function estimator), или REINFORCE. Для авторегрессионной политики

$$\log p_\theta(\pi | x) = \sum_{t=1}^T \log p_\theta(\pi_t | \pi_{<t}, x),$$

поэтому градиент содержит вклад каждого шага построения решения.

А.5. КОНТРОЛЬНАЯ ПЕРЕМЕННАЯ И ОЦЕНКА ГРАДИЕНТА

К стоимости можно вычесть базовую оценку $b(x)$ (baseline), которая не зависит от выбранной последовательности π . Для такой оценки

$$\begin{aligned} \mathbb{E}_{\pi \sim p_\theta} [b(x) \nabla_\theta \log p_\theta(\pi | x)] &= b(x) \sum_{\pi} p_\theta(\pi | x) \nabla_\theta \log p_\theta(\pi | x) \\ &= b(x) \nabla_\theta \sum_{\pi} p_\theta(\pi | x) \\ &= b(x) \nabla_\theta 1 = 0. \end{aligned}$$

Следовательно,

$$\nabla_\theta J = \mathbb{E}_{\pi \sim p_\theta} [(L(\pi | x) - b(x)) \nabla_\theta \log p_\theta(\pi | x)].$$

Вычитание базовой оценки не меняет математическое ожидание градиента и может уменьшать его дисперсию.

Для M независимо сгенерированных решений используется оценка

$$\widehat{\nabla_\theta J} = \frac{1}{M} \sum_{j=1}^M (L_j - b_j) \nabla_\theta \log p_\theta(\pi^{(j)} | x).$$

Ей соответствует суррогатная функция потерь

$$\hat{\mathcal{L}}_{\text{PG}} = \frac{1}{M} \sum_{j=1}^M \text{stopgrad}(L_j - b_j) \log p_{\theta}(\pi^{(j)} | x).$$

При минимизации стоимости коэффициент $L_j - b_j$ входит с положительным знаком. Если стоимость маршрута ниже базовой оценки, коэффициент отрицателен, и шаг градиентного спуска увеличивает вероятность маршрута.

В качестве базовой оценки используют:

- скользящее среднее стоимости по предыдущим пакетам;
- критик (critic), оценивающий условное ожидание стоимости по входу x ;
- стоимость маршрута, построенного детерминированной жадной политикой;
- среднее по остальным примерам (leave-one-out), вычисленное для примера j по независимо сгенерированным маршрутам.

Для несмещенности базовая оценка для примера j не должна зависеть от случайного действия $\pi^{(j)}$ при фиксированном x . Если она зависит от обучаемых параметров, градиент через нее в слагаемом градиента политики блокируется. Критик при этом обучается отдельной регрессионной функцией потерь. Среднее, включающее стоимость текущего маршрута L_j , зависит от $\pi^{(j)}$ и в общем случае вносит смещение.¹²

А.6. ВЫЧИСЛИТЕЛЬНЫЕ СВОЙСТВА И ОГРАНИЧЕНИЯ

Для задачи коммивояжера, где декодер выполняет $T = n$ шагов, обновление состояний LSTM требует $O(nd^2)$ операций, а вычисление pointer-attention по всем элементам на каждом шаге - $O(n^2d)$. Поэтому суммарная сложность кодирования и декодирования имеет порядок

$$O(nd^2 + n^2d).$$

Обученная модель строит приближенное решение для распределения задач, представленного в обучающей выборке. Оптимальность маршрута не гарантируется. Качество может снижаться при изменении числа элементов, распределения координат или ограничений задачи. Точные методы дают гарантию оптимальности. Приближенные алгоритмы могут давать гарантию на качество приближения. Для обученной политики таких гарантий обычно нет.

¹² Bello et al. Neural Combinatorial Optimization with Reinforcement Learning

Дополнительная литература

- [MIT 6.7960, Lecture 5: Graph Neural Networks](#) - общая схема передачи сообщений, постановки задач и агрегация представлений.
- [Stanford CS224W](#) - представления вершин, GNN, GraphSAGE, GAT, обучение и агрегация.
- [A Gentle Introduction to Graph Neural Networks](#) - визуальное объяснение графовых задач и обмена сообщениями.
- [Berkeley CS285: Policy Gradients](#) - вывод оценки на основе логарифмической производной и базовой оценки.